Speech Recognition System

Research, Implementation & Evaluation Project

Phase 1: Research and Analysis (Deadline Week 6)

Objective

Students are required to select 2 academic research papers **At least one** focused on Arabic Automatic Speech Recognition (Arabic ASR) systems. The purpose of this phase is to conduct in-depth analysis, critical evaluation, and comparative study of approaches, methodologies, and performance in Arabic speech recognition.

Team Size: 2-3 students per team (maximum 3 students)

Instructions

1. Paper Selection Requirements

Students must choose TWO research papers that meet ALL of the following criteria:

- Focus specifically on Speech recognition (Modern Standard Arabic, dialectal Arabic, or code-switching scenarios)
- Published in reputable venues: IEEE, ACL, ICASSP, NeurIPS, ICML, INTERSPEECH, EMNLP, NAACL, or equivalent journals/conferences
- Published in 2019 or later (to ensure current methodologies)
- Papers should ideally use different approaches or architectures for meaningful comparison

2. Required Analysis for Each Paper

A. Summary and Context

- Main topic and research objectives
- Specific problem addressed in ASR (e.g., dialectal variation, diacritization, code-switching)
- Key contributions and innovations
- Significance and potential impact on Arabic speech technology

B. Technical Architecture & Methodology

- Model architecture details (e.g., Transformer, Conformer, RNN-T, Wav2Vec 2.0, Whisper adaptations)
- Feature extraction methods (MFCC, Mel-spectrogram, raw waveform, etc.)
- Acoustic modeling approach
- Language modeling integration (if applicable)
- Training methodology: loss functions, optimization algorithms, learning rate schedules
- Special techniques for Arabic (diacritization handling, morphological preprocessing, etc.)
- Frameworks and tools used (PyTorch, TensorFlow, Kaldi, ESPnet, etc.)

C. Dataset and Experimental Design

- Datasets used: name, size, number of speakers, Arabic variety/dialect, audio duration
- Common Arabic ASR datasets: MGB-2, MGB-3, Common Voice Arabic, QASR, Shami, etc.
- Data preprocessing and augmentation techniques
- Train/validation/test split methodology
- Evaluation metrics: Word Error Rate (WER), Character Error Rate (CER), dialect-specific metrics
- Cross-dialect evaluation (if applicable)

D. Results and Performance Analysis

- Quantitative results: WER, CER, and other reported metrics
- Performance comparison with baseline and state-of-the-art models
- Performance across different dialects or varieties
- Notable improvements or innovations demonstrated
- Computational efficiency: model size, inference speed, training time

E. Bias and Fairness Analysis

Critical evaluation of potential biases in the Arabic ASR system:

- Dialect bias: Does the model perform significantly better on certain Arabic dialects (e.g., MSA vs. Egyptian vs. Gulf vs. Levantine)?
- Gender bias: Are there performance disparities between male and female speakers?
- Age bias: How does the system perform across different age groups?
- Socioeconomic and educational factors: Does accent or speech patterns associated with education level affect performance?
- Code-switching bias: How well does it handle Arabic-English or Arabic-French mixing?
- Dataset representation: Is the training data diverse and representative of various Arabic-speaking populations?
- Did the authors address or acknowledge any fairness concerns in their work?

F. Strengths (Pros)

Identify and explain the major advantages and strong points of the approach.

G. Limitations and Weaknesses (Cons)

Critical analysis of shortcomings, constraints, and areas for improvement.

3. Comparative Analysis

After thoroughly analyzing both papers, provide a comprehensive comparative discussion covering:

- Architectural differences: How do the model architectures differ fundamentally?
- Dataset differences: Variations in dataset size, dialect coverage, and quality
- Performance comparison: Which approach achieves better results? Under what conditions?
- Computational efficiency: Compare model complexity, training time, inference speed, resource requirements
- Practical deployment considerations: Which is more suitable for real-world applications?
- Robustness to linguistic challenges: Which handles dialectal variation, diacritization, morphology better?
- Bias and fairness: Which paper addresses fairness concerns more thoroughly?
- Overall assessment: Strengths and weaknesses of each approach

4. Citation and Academic Integrity Requirements

- **Use IEEE citation style for all references**
- Include a complete bibliography/references section at the end of your report
- Properly cite all sources: papers, datasets, frameworks, and any external resources used
- Direct quotes must be in quotation marks with page numbers
- Paraphrase properly - do not copy-paste text without attribution
- **Zero tolerance for plagiarism - any plagiarism will result in zero marks for the entire project**

Phase 2: Implementation and Presentation (Deadline Week 12)

Objective

Each team will implement ONE of the ASR papers from Phase 1 as a fully functional in any programming language with a working demonstration and comprehensive evaluation.

Implementation Requirements

1. Full Implementation (Not Conceptual)

You must create a WORKING ASR system with actual code and functionality.

A. Programming and Framework Examples

- Language: Python 3.8 or higher
- Deep Learning Framework: PyTorch or TensorFlow
- Recommended libraries: librosa, torchaudio, transformers, ESPnet, or similar
- Code must be well-structured, documented, and follow Python best practices

B. Dataset Requirements

You MUST use a real, substantial speech dataset. Suggested options:

- Common Voice Arabic (Mozilla) - open, multi-dialectal
- MGB-2 (Multi-Genre Broadcast 2) - dialectal Arabic
- MGB-3 (Egyptian dialect)
- QASR (Qatari Arabic Speech Recognition)
- Shami (Levantine dialect)
- CSASR (Code-Switched Arabic Speech Recognition) - if applicable
- Minimum requirement: At least 10 hours of speech data

C. Implementation Components

Your implementation must include all of the following:

1. Data Preprocessing Pipeline

- Audio loading and resampling
- Feature extraction (MFCC, Mel-spectrogram, or raw waveform)
- Normalization and augmentation
- Text preprocessing (tokenization, diacritics handling if applicable)

2. Model Architecture

- Implement or adapt the core architecture from your selected paper
- You may use pre-trained models (Wav2Vec2, Whisper) and fine-tune them for Arabic
- Or implement from scratch if feasible

3. Training Module

- Training loop with proper loss calculation

- Validation monitoring
- Checkpointing and model saving
- Learning rate scheduling

4. Inference/Decoding Module

- Real-time or batch transcription capability
- Beam search or greedy decoding
- Language model integration (optional but recommended)

5. Evaluation Module

- Calculate WER (Word Error Rate) and CER (Character Error Rate)
- Generate detailed error analysis
- Per-dialect performance breakdown (if using multi-dialect data)

6. User Interface/Demo Application

- Create a simple interface for demonstration (can be Gradio, Streamlit, or Flask web app)
- Allow users to upload audio files or record speech
- Display transcription results
- Show confidence scores or alternative hypotheses (optional)

2. Presentation Requirements

Each team will present their work in a 10-minute presentation covering:

- Paper selection rationale (2 minutes)
- Key findings from Phase 1 comparative analysis (1 minute)
- Implementation approach and architecture (2 minutes)
- Dataset and preprocessing pipeline (1 minute)
- Live demonstration of the working system (3 minutes)
- Results, evaluation, and challenges encountered (1 minute)

Presentation Guidelines:

- All team members must participate in the presentation
- Include architecture diagrams, flowcharts, or visualizations
- Show actual demo - not just screenshots
- Prepare for Q&A session (5 minutes)

Deliverables Summery

Phase 1:

- Written report (PDF) covering all required sections
- Properly formatted with IEEE citations
- 10-15 pages recommended length for each paper

Phase 2:

- Complete codebase (GitHub repository or ZIP file)
- Trained model weights or checkpoints
- Summary and documentation
- Presentation slides
- Demo video (5 minutes max) showing system functionality